

Sistem Monitoring Keamanan: Deteksi Plat Nomor dan Pengenalan Wajah yang Terintegrasi

Z. M. Salma., F. F. G. Wicaksono., M. R. Afkarean., R. N. Faruqi

Abstrak— Sistem monitoring keamanan berbasis teknologi computer vision telah dirancang untuk mendeteksi dan mengenali plat nomor kendaraan serta wajah pengemudi secara otomatis di area parkir. Sistem ini menggunakan algoritma Haar Cascade untuk deteksi wajah, Optical Character Recognition (OCR) untuk membaca plat nomor, dan metode cosine similarity untuk memverifikasi kecocokan data pengemudi. Dengan metode verifikasi ganda ini, sistem mampu meningkatkan keamanan parkir dengan signifikan, meminimalkan potensi pencurian, dan mengurangi ketergantungan pada pengawasan manual. Meskipun hasil menunjukkan tingkat akurasi yang tinggi, tantangan seperti variasi pencahayaan dan sudut kamera tetap menjadi perhatian utama untuk pengembangan lebih lanjut. Sistem ini berpotensi diaplikasikan secara luas untuk meningkatkan keamanan di berbagai fasilitas publik.

Kata Kunci— Computer vision, deteksi plat nomor, pengenalan wajah, Haar Cascade, Optical Character Recognition (OCR), cosine similarity, sistem keamanan parkir.

Abstract— A security monitoring system based on computer vision technology has been designed to automatically detect and recognize vehicle license plates and driver faces in parking areas. This system employs the Haar Cascade algorithm for face detection, Optical Character Recognition (OCR) for reading license plates, and cosine similarity for verifying driver data. By implementing this dual verification method, the system significantly enhances parking security, minimizes theft risks, and reduces reliance on manual supervision. Although the results demonstrate high accuracy, challenges such as lighting variations and camera angles remain critical for further development. This system has the potential to be widely applied to improve security in various public facilities.

Keywords— Computer vision, license plate detection, face recognition, Haar Cascade, Optical Character Recognition (OCR), cosine similarity, parking security system.

I. PENDAHULUAN

KASUS pencurian kendaraan bermotor, khususnya sepeda motor, masih menjadi salah satu kejahatan yang marak terjadi di Indonesia. Data dari Pusat Informasi Kriminal Nasional (Pusiknas) Bareskrim Polri menunjukkan bahwa pencurian kendaraan bermotor roda dua mencatat angka hingga 15.358 kasus per Oktober tahun 2024 [1]. Pada tahun 2022, dalam tiga minggu pada bulan Mei saja, terdapat sekitar 766 kasus pencurian sepeda motor yang terjadi di seluruh Indonesia [2]. Berbagai modus operandi, seperti perusakan kunci atau manipulasi kepemilikan kendaraan, semakin memperparah situasi ini.

Meskipun keberadaan petugas parkir dapat memberikan pengawasan untuk mencegah tindakan kriminal, dalam praktiknya, kehadiran petugas di lapangan tidak selalu konstan. Hal ini menciptakan celah keamanan yang dapat dimanfaatkan oleh pihak-pihak yang berniat melakukan tindakan pencurian. Selain itu, kurangnya pengawasan yang optimal sering kali menyebabkan ketidaktepatan dalam memverifikasi kepemilikan kendaraan saat proses keluar dari area parkir, sehingga kendaraan yang bukan miliknya dapat dibawa keluar oleh orang lain.

Dengan meningkatnya angka pencurian dan keterbatasan sumber daya manusia untuk pengawasan konstan di area parkir, solusi berbasis teknologi *computer vision* muncul sebagai pendekatan inovatif dalam menjaga keamanan kendaraan. *Computer vision* kini memungkinkan pengenalan otomatis terhadap plat nomor kendaraan dan wajah pemiliknya, yang berfungsi sebagai metode verifikasi ganda dalam upaya meningkatkan keamanan parkir [3]. Sistem keamanan berbasis pengenalan wajah ini berfungsi untuk memverifikasi identitas pemilik kendaraan secara otomatis, sehingga memastikan bahwa hanya individu yang sama yang memasuki dan meninggalkan area parkir dengan kendaraan tersebut. Selain itu, integrasi antara teknologi pengenalan wajah dan plat nomor dapat memperkuat akurasi dalam proses verifikasi identitas pemilik kendaraan dan menambah lapisan keamanan ekstra di area parkir.

Teknologi ini juga memungkinkan sistem untuk bekerja secara otomatis dan efisien dalam melakukan verifikasi dan pengawasan, sehingga mengurangi ketergantungan pada petugas parkir dan meningkatkan keamanan di area publik [4]. Dengan penggunaan teknologi pengenalan wajah dan plat nomor yang saling terintegrasi, sistem ini mampu menghadirkan perlindungan yang lebih baik dan secara langsung mengurangi risiko pencurian kendaraan. Hal ini juga memberikan kenyamanan lebih bagi para pengguna area parkir yang mengharapkan keamanan lebih tinggi di tengah meningkatnya angka kriminalitas.

II. TINJAUAN PUSTAKA

A. Penelitian Sebelumnya

Berbagai pendekatan telah dikembangkan untuk meningkatkan kinerja sistem parkir berbasis computer vision. Pendekatan-pendekatan tersebut mencakup penggunaan teknik segmentasi citra, deteksi objek berbasis algoritma tradisional

maupun jaringan saraf tiruan, Selain itu, beberapa penelitian juga memanfaatkan kombinasi metode berbasis machine learning dan deep learning untuk mengatasi tantangan seperti variasi pencahayaan, sudut pandang, dan keberadaan noise pada citra. Keberagaman metode ini mencerminkan kompleksitas masalah dan kebutuhan untuk menemukan solusi yang optimal dalam berbagai kondisi.

Penelitian oleh Shejina V [5], studinya mengusulkan sistem keamanan berbasis pengenalan wajah yang dirancang untuk diintegrasikan langsung ke dalam kendaraan. Sistem ini menggunakan modul Raspberry Pi dengan algoritma deteksi wajah berbasis Viola-Jones dan pengenalan menggunakan Local Binary Patterns Histograms (LBPH). Kamera tersembunyi mengambil gambar pengemudi saat kunci dimasukkan ke lubang kunci, dan gambar tersebut dianalisis untuk menentukan apakah pengemudi adalah pengguna yang sah atau tidak. Jika kendaraan digunakan oleh pengemudi tidak sah, sistem secara otomatis mengirimkan pemberitahuan berupa SMS berisi gambar pengemudi dan lokasi kendaraan kepada pemiliknya. Studi ini memberikan wawasan penting tentang penggunaan teknologi berbasis visi komputer untuk meningkatkan keamanan kendaraan secara real-time.

Penelitian lain yang memiliki mekanisme sama dengan yang akan kami buat adalah milik Truong dkk [6]. Penelitian ini menyoroti integrasi teknologi cerdas dalam sistem parkir mengusulkan penggunaan teknologi pengenalan citra video sebagai solusi anti-pencurian. Sistem ini dirancang untuk mendeteksi keberadaan pengemudi yang bukan pemilik kendaraan melalui analisis pengenalan wajah dan pelat nomor kendaraan. Gambar kendaraan diambil secara otomatis oleh kamera di pintu masuk area parkir, lalu diproses menggunakan convolutional network berbasis deep learning untuk pengenalan wajah, serta algoritma pengenalan karakter atau Optical Character Recognition untuk pelat nomor. Hasil pengenalan ini dibandingkan untuk memverifikasi kecocokan antara wajah pengemudi dan data pelat nomor yang terdaftar. Sistem ini dapat diterapkan di tempat parkir umum maupun garasi pribadi, memberikan peringatan dini jika terjadi potensi pencurian. Meskipun menunjukkan hasil yang menjanjikan, penelitian ini juga mencatat adanya keterbatasan, seperti kesulitan mendeteksi wajah berukuran kecil, obyek yang terhalang bayangan, atau gambar yang diambil melalui kaca, sehingga diperlukan studi lanjutan untuk mengoptimalkan faktor sudut, pencahayaan, dan jenis kamera yang sesuai.

Penelitian oleh Africa dkk menggunakan teknologi visi komputer dengan algoritma utama yang melibatkan pengolahan citra dasar dan pengenalan karakter [7]. Pada tahap awal, gambar pelat nomor diproses untuk memisahkan spektrum warna (RGB) guna menghasilkan gambar skala abu-abu. Frekuensi tinggi pada gambar pelat nomor diidentifikasi menggunakan filter Gaussian, yang memungkinkan pelat nomor dapat dibedakan dari elemen lain di gambar berdasarkan intensitas piksel. Setelah itu, pengenalan karakter dilakukan dengan algoritma pencocokan karakter, di mana gambar hitam-putih dari pelat nomor digunakan untuk mengidentifikasi huruf dan angka dengan mencocokkan simbol pada posisi tertentu.

Pendekatan ini memungkinkan identifikasi pelat nomor secara otomatis dengan akurasi tinggi.

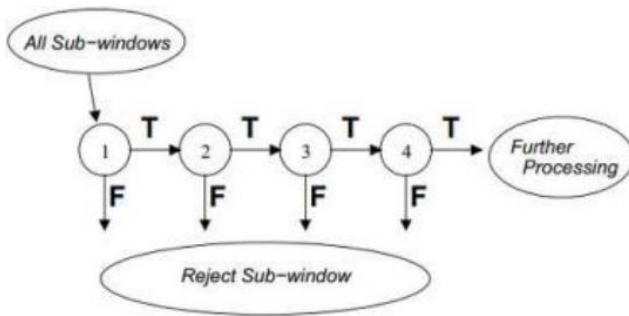
Namun, Africa dkk menghadapi beberapa kesulitan selama pengembangan sistem ini. Tantangan utama meliputi pengolahan gambar yang terpengaruh oleh kondisi pencahayaan, sudut pengambilan gambar, dan adanya objek lain yang menghalangi pandangan ke pelat nomor. Selain itu, pelat nomor yang buram atau rusak akibat usia kendaraan juga dapat menurunkan akurasi sistem. Kendala lain mencakup kebutuhan akan komputasi yang cukup tinggi untuk menjalankan algoritma secara real-time, yang menuntut optimasi perangkat keras agar sistem dapat bekerja secara efisien di lapangan.

Berdasarkan berbagai penelitian terdahulu, dapat disimpulkan bahwa teknologi visi komputer telah memberikan kontribusi signifikan dalam pengembangan sistem parkir dan keamanan kendaraan. Beragam pendekatan, mulai dari pengenalan wajah hingga pengenalan pelat nomor berbasis pembelajaran mendalam, telah menunjukkan efektivitasnya dalam meningkatkan efisiensi pengelolaan parkir dan pencegahan pencurian kendaraan. Namun, tantangan seperti variasi kondisi pencahayaan, sudut pandang kamera, dan kebutuhan perangkat keras yang mumpuni masih menjadi perhatian utama yang perlu diatasi dalam implementasi di dunia nyata. Dengan perkembangan teknologi yang terus berlanjut, penelitian-penelitian ini memberikan landasan yang kuat untuk mengembangkan sistem yang lebih canggih, efisien, dan adaptif di masa depan.

B. Face Recognition

a. Haar Cascade Classifier

Haar Cascade Classifier adalah algoritma yang menggunakan teknik pembelajaran mesin untuk mendeteksi objek dalam citra, khususnya wajah manusia yang ditemukan oleh Paul Viola dan Michael Jones pada tahun 2001. Haar Cascade bekerja dengan mengekstraksi fitur dari gambar menggunakan sebuah filter yang disebut fitur Haar, yang mirip dengan konsep kernel konvolusional. Filter ini memindai satu area gambar pada satu waktu. Untuk setiap area, nilai intensitas piksel di bagian hitam dan putih dijumlahkan, lalu dihitung selisih antara kedua jumlah tersebut. Selisih ini menjadi nilai fitur yang diekstraksi. Proses ini diikuti dengan klasifikasi bertingkat, di mana setiap tingkatan memfilter subcitra yang tidak memenuhi kriteria fitur Haar tertentu. Pada tahap awal, klasifikasi menyisakan sekitar 50% subcitra untuk diproses lebih lanjut [8].



Gambar 2.1 Klasifikasi Bertingkat Fitur Haar [8]

Pada setiap tingkatan klasifikasi berikutnya, kriteria yang lebih spesifik diterapkan, yang didukung oleh fitur tambahan dari citra. Hasil klasifikasi di setiap tahap berupa "True" (T) untuk subcitra yang memenuhi fitur tertentu dan "False" (F) untuk yang tidak. Seiring meningkatnya tingkatan klasifikasi, jumlah subcitra yang lolos semakin sedikit hingga tersisa hanya sejumlah kecil subcitra yang relevan. Metode ini memastikan efisiensi deteksi objek, seperti wajah, dengan fokus pada area gambar yang memiliki peluang lebih tinggi untuk mengandung fitur yang dicari [8].

b. Histogram of Oriented Gradients(HOG)

HOG adalah teknik yang mengandalkan informasi tentang orientasi dan kekuatan tepi dalam citra. Proses ini dimulai dengan menghitung gradien dari citra untuk menentukan arah dan magnitudo tepi. Citra kemudian dibagi menjadi blok kecil, dan histogram dari arah gradien dihitung dalam setiap blok. Histogram ini kemudian dinormalisasi untuk mengurangi efek pencahayaan dan kontras yang tidak diinginkan. Hasil akhir adalah deskriptor fitur yang dapat digunakan untuk pelatihan model klasifikasi, seperti Support Vector Machine (SVM) atau Random Forest [9], [10].

Penelitian lain mengintegrasikan HOG dengan algoritma pembelajaran mesin seperti Random Forest dan SVM, menunjukkan bahwa kombinasi ini dapat meningkatkan akurasi pengenalan wajah hingga 99,54% [11].

C. Similarity Menggunakan Metode Cosine Similarity

Cosine similarity adalah metode yang digunakan untuk mengukur kesamaan antara dua vektor dalam ruang multidimensi dengan menghitung sudut di antara mereka. Nilai cosine similarity berkisar antara -1 hingga 1, di mana 1 menunjukkan bahwa kedua vektor identik, 0 menunjukkan tidak ada kesamaan, dan -1 menunjukkan bahwa kedua vektor berlawanan. Dalam konteks pengenalan wajah, cosine similarity sangat berguna karena dapat mengukur

seberapa mirip dua wajah berdasarkan representasi fitur yang diekstraksi dari gambar wajah tersebut [12].

Dalam pengenalan wajah, metode cosine similarity sering diterapkan setelah proses ekstraksi fitur dari gambar wajah. Fitur-fitur ini dapat diekstraksi menggunakan berbagai teknik seperti *Histogram of Oriented Gradients* (HOG). Setelah fitur diekstraksi, *cosine similarity* digunakan untuk membandingkan vektor fitur dari gambar yang berbeda. Misalnya, dalam sebuah penelitian, cosine similarity digunakan untuk mencocokkan kesamaan gambar wajah dalam basis data standar seperti ORL dan Yale [13].

$$\cos Sim(d_j, q_k) = \frac{\sum_{i=1}^n (td_{ij} \times t_{qik})}{\sqrt{\sum_{i=1}^n td_{ij}^2 \times \sum_{i=1}^n t_{qik}^2}}$$

Keterangan :

$\cos Sim(d_j, q_k)$: tingkat kesamaan dengan *query* tertentu

td_{ij} : *term* ke-i dalam vektor untuk dokumen ke-j

t_{qik} : *term* ke-i dalam vektor untuk *query* ke-k

n : jumlah *term* yang unik dalam data set

Gambar 2.2 Rumus Metode Cosine Similarity [12]

Pada rumus di gambar 2 didapat dengan membagi hasil kali dot dengan magnitudo dari kedua vektor, kita mendapatkan nilai cosine similarity yang mencerminkan sudut antara kedua vektor tersebut. Semakin kecil sudutnya, semakin besar nilai cosine similarity.

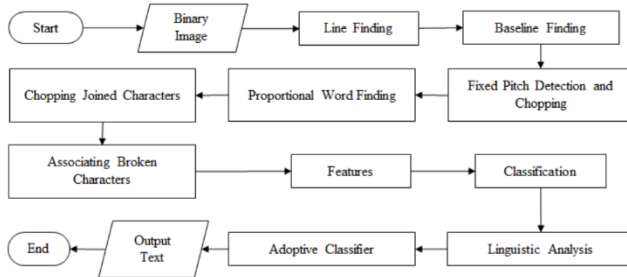
Cosine similarity memiliki beberapa keunggulan dalam pengenalan wajah, di antaranya adalah ketahanan terhadap perubahan pencahayaan, yang membuatnya lebih robust dibandingkan dengan metode jarak lainnya seperti Euclidean distance. Selain itu, metode ini efisien dalam komputasi, sehingga cocok untuk aplikasi waktu nyata yang memerlukan kecepatan. *Cosine similarity* juga sangat efektif dalam menangani data berdimensi tinggi, seperti representasi fitur wajah, karena dapat mengukur kemiripan antara vektor-vektor fitur meskipun dimensi datanya sangat besar, tanpa terpengaruh oleh perbedaan skala antara fitur-fitur tersebut [14].

D. Plat detection using Optical Character Recognition (OCR)

Optical Character Recognition (OCR) adalah teknologi yang digunakan untuk mengonversi teks yang dicetak atau ditulis tangan dari gambar atau dokumen yang dipindai menjadi format yang dapat dibaca oleh mesin. Proses ini sangat penting dalam digitalisasi dokumen, meningkatkan produktivitas, dan aksesibilitas data, serta untuk pelestarian dokumen bersejarah. OCR memungkinkan ekstraksi teks

otomatis, yang sangat berguna dalam berbagai aplikasi seperti pengenalan plat nomor kendaraan, pengolahan dokumen, dan sistem pencarian informasi [15], [16].

Untuk Menggunakan OCR dapat dilakukan menggunakan *pytesseract* yaitu sebuah library pada Python. Pytesseract adalah sebuah alat yang berfungsi sebagai antarmuka untuk mesin Tesseract-OCR buatan Google. Alat ini mendukung berbagai format gambar seperti JPEG dan PNG. Pytesseract sering digunakan untuk mempermudah proses ekstraksi teks dari gambar digital [17].



Gambar 2.3 Flowchart Pytesseract [18]

Algoritma Pytesseract terlihat pada gambar 3 dimulai dengan mengolah gambar biner melalui tahapan line finding untuk menemukan baris teks, baseline fitting untuk menentukan garis dasar menggunakan quadratic spline, dan pitch detection and chopping untuk memisahkan karakter. Selanjutnya, dilakukan proportional word finding untuk menentukan jarak antar kata, serta chopping joined characters untuk memotong karakter yang saling terhubung dan associating broken characters untuk menyatukan karakter terpisah menggunakan algoritma A*. Ekstraksi fitur dilakukan dengan membandingkan karakter dengan prototipe data, diikuti oleh klasifikasi berdasarkan vektor prototipe. Tahapan akhir mencakup analisis linguistik sesuai bahasa yang diatur melalui parameter “lang” dan adaptive classifier yang membedakan huruf besar-kecil, menormalkan tinggi karakter, serta meningkatkan ketahanan terhadap noise [19].

Pytesseract mempermudah implementasi OCR dalam program Python dengan hasil pembacaan yang presisi. Fitur bawaan seperti pengolahan gambar, ekstraksi fitur, dan segmentasi teks meningkatkan akurasi serta efisiensi. Modul linguistiknya mendukung pengklasifikasian kata berdasarkan bahasa yang dipilih. Sebagai perangkat open source, Pytesseract memungkinkan pengembang global berkontribusi pada pembaruan dan inovasi. Dokumentasinya yang jelas juga memudahkan pengguna mempelajarinya [18].

III. METODE PENELITIAN

A. Flowchart

Berdasarkan gambar 6.1 yang terdapat pada lampiran, penjelasan *flowchart* dipaparkan sebagai berikut.

1. Mulai

Proses dimulai dengan menjalankan aplikasi Sistem *Monitoring* Keamanan. Pada tahap ini, program akan memastikan semua komponen perangkat keras seperti kamera untuk wajah dan plat nomor telah terhubung dengan baik ke sistem. Selain itu, perangkat lunak pendukung seperti algoritma pengenalan wajah dan *Optical Character Recognition* (OCR) untuk membaca plat nomor akan siap diinisialisasi. Tahap ini juga memastikan integritas database yang akan digunakan untuk mencatat atau mencari data kendaraan.

2. Inisialisasi Sistem

Sistem menginisialisasi semua komponen utama, termasuk modul pengenalan wajah dan OCR. Inisialisasi meliputi:

1. Kamera: Sistem memverifikasi bahwa kamera dapat menangkap gambar secara real-time, baik untuk wajah maupun plat nomor.
2. Model Deteksi Wajah: Menggunakan algoritma *Haar Cascade Classifier* (dari OpenCV) untuk mendeteksi wajah secara langsung dari input kamera.
3. Model Deteksi Plat Nomor: Menggunakan algoritma *Haar Cascade Classifier* khusus untuk mendeteksi area plat nomor pada gambar kendaraan.
4. Preprocessing Data: Sistem mengatur parameter awal seperti resolusi gambar, skema warna (misalnya, konversi ke grayscale), dan pengaturan ROI (Region of Interest) untuk area tertentu.

Tujuan inisialisasi adalah memastikan bahwa semua modul siap bekerja dan dapat menangani data yang masuk tanpa kesalahan.

3. Pilih Menu

Setelah inisialisasi selesai, program menampilkan tiga opsi menu utama:

1. Masuk: Untuk mencatat kendaraan masuk beserta data wajah dan plat nomor.
2. Keluar: Untuk mencatat kendaraan keluar dengan memverifikasi data wajah dan plat nomor.
3. Cari Data: Untuk mencari data kendaraan tertentu berdasarkan teks plat nomor yang dimasukkan pengguna.

Operator akan memilih salah satu dari menu tersebut untuk melanjutkan proses sesuai dengan kebutuhan.

4. Capture Gambar Wajah (Masuk)

Ketika operator memilih menu Masuk, sistem pertama-tama mengaktifkan kamera untuk mengambil gambar wajah pengemudi. Proses ini dilakukan dengan langkah-langkah berikut:

1. Deteksi Wajah

Sistem menggunakan *Haar Cascade Classifier* untuk mendeteksi wajah di area gambar. Algoritma ini bekerja dengan mencocokkan fitur wajah (seperti mata, hidung, dan mulut) dengan pola yang telah dilatih sebelumnya.

2. ROI Wajah

Setelah mendeteksi wajah, sistem menentukan ROI untuk mengekstrak bagian wajah dari gambar.

3. Peningkatan Gambar

Sistem mengubah ROI ke grayscale untuk mempermudah penyimpanan dan proses selanjutnya.

4. Penyimpanan Gambar Wajah

Gambar wajah yang diambil disimpan ke dalam memori sementara untuk digabungkan dengan data plat nomor. Jika kamera tidak berhasil mendeteksi wajah, sistem akan memberikan peringatan agar pengguna memposisikan wajah mereka dengan benar di depan kamera.

5. Capture Gambar Plat Nomor (Masuk)

Setelah gambar wajah berhasil ditangkap, kamera kedua diaktifkan untuk menangkap gambar plat nomor kendaraan. Langkah-langkahnya meliputi:

1. Deteksi Plat Nomor: Algoritma *Haar Cascade Classifier* mendeteksi posisi plat nomor pada kendaraan.
2. ROI Plat Nomor: Sistem mengekstrak area plat nomor berdasarkan deteksi yang dilakukan, yang nantinya digunakan untuk OCR.
3. Validasi Hasil: Sistem memastikan bahwa area yang ditangkap memang benar merupakan plat nomor. Jika tidak valid, sistem akan meminta pengguna untuk menyesuaikan posisi kendaraan.

6. Deteksi dan Preprocess Plat Nomor (Masuk)

Proses ini melibatkan deteksi area plat nomor dan preprocessing gambar untuk meningkatkan akurasi OCR. Langkah-langkah yang dilakukan:

1. Konversi Grayscale: Plat nomor dikonversi ke skala abu-abu (grayscale) untuk menyederhanakan data gambar.
2. Peningkatan Kontras: *Histogram Equalization* digunakan untuk menyesuaikan kontras gambar agar huruf dan angka lebih mudah dibaca.
3. Penghilangan Noise: Filter Gaussian diterapkan untuk mengurangi noise pada gambar, seperti bintik-bintik atau gangguan lain yang bisa mempengaruhi OCR.
4. Thresholding: Teknik *Adaptive Thresholding* digunakan untuk mengubah gambar menjadi biner (hitam-putih), mempermudah proses OCR dalam mengenali karakter.

7. Ekstraksi Teks Plat Nomor (Masuk)

Pada tahap ini, algoritma OCR diterapkan untuk mengekstrak teks dari gambar plat nomor. Sistem menggunakan pustaka Tesseract OCR yang bekerja sebagai berikut:

1. Segmentasi Karakter: Plat nomor dibagi menjadi karakter individual (huruf dan angka).
2. Pengenalan Karakter: Algoritma OCR mengenali setiap karakter berdasarkan pola yang telah dilatih sebelumnya.
3. Validasi Teks: Sistem memeriksa apakah hasil OCR sesuai dengan format plat nomor yang diharapkan. Misalnya, plat nomor Indonesia memiliki format "B 1234 XYZ".

Hasil berupa teks plat nomor disimpan dalam sistem untuk proses lebih lanjut.

8. Simpan Data Masuk

Setelah wajah dan plat nomor berhasil diproses, data berikut disimpan dalam database:

- Gambar Wajah: Disimpan dalam format gambar yang dikompresi untuk efisiensi ruang.
- Teks Plat Nomor: Disimpan dalam bentuk string yang terhubung dengan data wajah.
- Waktu Masuk: Ditambahkan untuk mencatat waktu kedatangan kendaraan.

Data ini berguna untuk proses validasi ketika kendaraan akan keluar atau untuk kebutuhan pencarian.

9. Capture Gambar Wajah (Keluar)

Pada menu Keluar, sistem kembali mengaktifkan kamera untuk menangkap gambar wajah pengemudi. Proses ini sama seperti pada tahap *Masuk*, tetapi hasilnya digunakan untuk validasi dengan data yang sudah tersimpan. Jika deteksi wajah gagal, pengguna akan diminta untuk mencoba lagi.

10. Capture Gambar Plat Nomor (Keluar)

Sama seperti proses *Masuk*, sistem menangkap gambar plat nomor kendaraan. Hasil gambar ini juga digunakan untuk validasi dengan data yang tersimpan di database.

11. Deteksi dan Preprocess Plat Nomor (Keluar)

Proses preprocessing dilakukan sama seperti pada tahap *Masuk*, dengan konversi ke grayscale, peningkatan kontras, penghilangan noise, dan thresholding. Tujuan utamanya adalah memastikan teks pada plat nomor dapat diekstrak dengan akurat.

12. Ekstraksi Teks Plat Nomor (Keluar)

OCR kembali digunakan untuk mengekstrak teks plat nomor dari gambar yang telah diproses. Hasilnya dibandingkan dengan data yang tersimpan di database untuk validasi.

13. Validasi Data Wajah dan Plat

Tahap ini membandingkan data wajah dan plat nomor dari input *Keluar* dengan data yang sudah tercatat pada tahap *Masuk*. Validasi dilakukan menggunakan algoritma *string comparison* untuk teks plat nomor dan algoritma *cosine similarity* untuk wajah, guna memastikan kemiripan gambar wajah antara data masuk dan keluar. Jika validasi berhasil, kendaraan diizinkan keluar. Jika tidak, sistem memberikan peringatan.

14. Hapus Data Entri

Jika validasi data berhasil, data kendaraan tersebut dihapus dari database, mencatat bahwa kendaraan tersebut sudah keluar. Ini bertujuan untuk menjaga konsistensi dan efisiensi database.

15. Tolak Kendaraan Keluar

Jika validasi gagal, sistem menolak kendaraan keluar dan memberikan peringatan kepada operator bahwa data wajah atau plat nomor tidak cocok dengan data yang tersimpan.

16. Masukkan Plat Nomor untuk Pencarian

Pada menu Cari Data, pengguna diminta memasukkan teks plat nomor secara manual. Teks ini digunakan untuk mencari data kendaraan di database.

17. Apakah Plat Ditemukan?

Sistem memeriksa database untuk mencari data yang sesuai dengan plat nomor yang dimasukkan. Jika ditemukan, data kendaraan ditampilkan. Jika tidak, sistem memberikan pesan bahwa plat nomor tersebut tidak terdaftar.

18. Menampilkan Data Masuk Kendaraan

Jika plat nomor ditemukan, sistem menampilkan data lengkap kendaraan, termasuk waktu masuk, gambar wajah, dan informasi lain yang relevan.

19. Tampilkan Pesan: Plat Tidak Ditemukan

Jika plat nomor tidak ditemukan, sistem memberikan pesan kesalahan untuk memberitahu pengguna bahwa data tidak tersedia.

20. Selesai

Proses berakhir dan sistem kembali ke menu utama untuk menunggu perintah atau input selanjutnya.

B. Alat dan Bahan

Dalam proyek ini, kami menggunakan berbagai alat dan bahan yang mendukung proses deteksi dan pengenalan plat nomor serta wajah. Alat utama adalah **kamera webcam** dengan spesifikasi resolusi **1080p**, yang digunakan untuk menangkap gambar plat nomor kendaraan. Kamera ini dipilih karena resolusinya yang tinggi, sehingga mampu menghasilkan gambar yang tajam dan memudahkan proses OCR (Optical Character Recognition). Untuk deteksi dan pengenalan wajah, kami menggunakan **kamera internal laptop** dengan resolusi **720p**. Kamera ini cukup memadai untuk mendeteksi fitur wajah menggunakan metode Haar Cascade dan ekstraksi fitur menggunakan Histogram of Oriented Gradients (HoG).

Perangkat komputasi yang digunakan adalah **laptop MSI** dengan prosesor **Intel Core i7** generasi terbaru dan kartu grafis **NVIDIA RTX 3060**. Spesifikasi ini memastikan bahwa semua proses, termasuk deteksi, cropping, preprocessing, dan OCR, dapat dijalankan dengan cepat dan efisien tanpa hambatan performa.

Dari sisi perangkat lunak, kami menggunakan **Python** sebagai bahasa pemrograman utama karena kemampuannya yang luas dalam pengolahan citra dan kompatibilitasnya dengan berbagai pustaka pendukung. Library yang digunakan meliputi:

1. **OpenCV**: Untuk pengolahan citra seperti deteksi objek, cropping, preprocessing, dan fitur Haar Cascade.
2. **NumPy**: Untuk manipulasi data numerik dan array yang diperlukan dalam pengolahan citra.
3. **Pytesseract**: Untuk membaca teks dari gambar plat nomor (OCR).

4. **dlib**: Untuk deteksi wajah dan ekstraksi fitur menggunakan metode HoG.
5. **Pandas**: Untuk menyimpan dan memanipulasi data masuk dan keluar dalam format tabel.
6. **Matplotlib**: Untuk visualisasi hasil deteksi dan proses pengolahan citra.

Kombinasi alat dan bahan ini dirancang untuk memberikan hasil yang optimal dalam sistem pengenalan plat nomor dan wajah.

B. Prosedur Percobaan

Plat Nomor Masuk

a. Pengambilan Gambar

- Gambar diambil menggunakan dua kamera: webcam untuk wajah dan kamera bawaan untuk plat nomor.
- Kode memeriksa bahwa kedua kamera berfungsi, lalu menangkap gambar setelah hitungan mundur 5 detik.
- Keterbatasan:
 - Tidak ada logika spesifik untuk memastikan warna plat (putih).
 - Tidak ada pengukuran jarak kamera ke plat dalam kode, tetapi hal ini dapat menyebabkan kegagalan deteksi jika plat terlalu jauh atau kecil di gambar.

b. Plat nomor di crop ukuran 240 x 60

- Plat nomor terdeteksi berdasarkan rasio aspek (1.5 hingga 6) dan area (>5000 piksel).
- Setelah bounding box ditemukan, cropping dilakukan dengan menambahkan padding 10% dari ukuran awal.
- Setelah cropping, ukuran hasilnya adalah 240 x 60 piksel, sesuai dengan proses resize dalam `detect_license_plate()`.

c. Preprocessing jadi biner (hitam putih)

- Plat nomor yang sudah dicrop di-preprocess menggunakan:
 - Grayscale.
 - Gaussian Blur untuk mengurangi noise.
 - Thresholding (Otsu atau Adaptive) untuk menghasilkan gambar biner (hitam putih).
- Ini dilakukan dalam fungsi `preprocess_license_plate()`.

d. Gambar disimpan

- Gambar preprocessed plat nomor disimpan dengan timestamp untuk referensi.
- Penamaan file mengikuti format: `preprocessed_license_entry_<timestamp>.jpg` atau `preprocessed_license_exit_<timestamp>.jpg`.

- e. Plat di OCR, Keluar Hasil, Hasil Disimpan dalam Data
 - Plat nomor yang sudah di-preprocess dikenali menggunakan OCR (pytesseract).
 - Konfigurasi OCR memastikan hanya huruf dan angka yang diproses.
 - Hasil teks diolah untuk mencocokkan pola plat (seperti AB 1234 CD).
 - Jika valid, hasil disimpan dalam dictionary entries di memori (dan file entries.pkl jika program diakhiri).

Plat Nomor Keluar

- a. Pengambilan gambar
 - Gambar diambil menggunakan dua kamera: webcam untuk wajah dan kamera bawaan untuk plat nomor.
 - Kode memeriksa bahwa kedua kamera berfungsi, lalu menangkap gambar setelah hitungan mundur 5 detik.
 - Keterbatasan:
 - Tidak ada logika spesifik untuk memastikan warna plat (putih).
 - Tidak ada pengukuran jarak kamera ke plat dalam kode, tetapi hal ini dapat menyebabkan kegagalan deteksi jika plat terlalu jauh atau kecil di gambar.
- b. Plat nomor di crop ukuran 240 x 60
 - Plat nomor terdeteksi berdasarkan rasio aspek (1.5 hingga 6) dan area (>5000 piksel).
 - Setelah bounding box ditemukan, cropping dilakukan dengan menambahkan padding 10% dari ukuran awal.
 - Setelah cropping, ukuran hasilnya adalah 240 x 60 piksel, sesuai dengan proses resize dalam detect_license_plate().
- c. preprocessing jadi biner (hitam putih)
 - Plat nomor yang sudah dicrop di-preprocess menggunakan:
 - Grayscale.
 - Gaussian Blur untuk mengurangi noise.
 - Thresholding (Otsu atau Adaptive) untuk menghasilkan gambar biner (hitam putih).
 - Ini dilakukan dalam fungsi preprocess_license_plate().
- d. Gambar disimpan
 - Gambar preprocessed plat nomor disimpan dengan timestamp untuk referensi.
 - Penamaan file mengikuti format: preprocess_license_entry_<timestamp>.jpg atau preprocess_license_exit_<timestamp>.jpg.

- e. Plat di OCR, Keluar Hasil, Hasil Disimpan dalam Data
 - OCR (pytesseract) membaca teks dari gambar plat.
 - Hasil OCR disesuaikan dengan pola plat (seperti AB 1234 CD) untuk memastikan validitas data.
- f. Mencocokkan hasil pembacaan ocr dengan data masuk
 - Hasil OCR dibandingkan dengan data di dictionary entries.
 - Jika plat ditemukan di data masuk, plat diperbolehkan keluar.
 - Jika tidak ditemukan, plat ditolak

Wajah Masuk

- a. Pengambilan gambar
 - Gambar wajah diambil menggunakan webcam.
 - Setelah hitungan mundur 5 detik, gambar diambil untuk diproses lebih lanjut.
 - Keterbatasan: Sistem tidak dapat memproses dua wajah sekaligus. Jika terdapat lebih dari satu wajah dalam gambar, proses akan dihentikan, dan pengguna diminta mengambil ulang gambar.
- b. Deteksi Wajah dengan Haar Cascade dan Crop
 - Wajah dideteksi menggunakan Haar Cascade. Bounding box ditentukan untuk area wajah.
 - Area wajah yang terdeteksi dicrop menjadi ukuran spesifik sesuai deteksi.
- c. Gambar disimpan
 - Gambar wajah hasil crop disimpan dalam format seperti face_entry_<timestamp>.jpg.

Wajah Keluar

- a. Pengambilan gambar
 - Proses sama seperti pengambilan gambar untuk wajah masuk.
 - Keterbatasan: Sistem tidak dapat memproses dua wajah sekaligus. Jika terdapat lebih dari satu wajah dalam gambar, proses akan dihentikan, dan pengguna diminta mengambil ulang gambar.
- b. Deteksi Wajah dengan Haar Cascade dan Crop
 - Wajah dideteksi menggunakan Haar Cascade. Bounding box ditentukan untuk area wajah.
 - Area wajah yang terdeteksi dicrop menjadi ukuran spesifik sesuai deteksi.
- c. Ekstraksi fitur bentuk wajah masuk dan keluar menggunakan HoG
 - Ekstraksi fitur dilakukan dengan Histogram of Oriented Gradients (HoG) pada gambar wajah masuk dan keluar.
 - HoG menghasilkan representasi fitur unik dari struktur wajah.

- d. Mencocokkan hasil ekstraksi fitur wajah masuk dan keluar
 - Fitur wajah yang diekstrak dibandingkan menggunakan metode pencocokan (misalnya, jarak Euclidean).
 - Jika fitur sesuai, wajah diperbolehkan keluar.
 - Jika tidak sesuai, wajah ditolak.

IV. HASIL DAN PEMBAHASAN

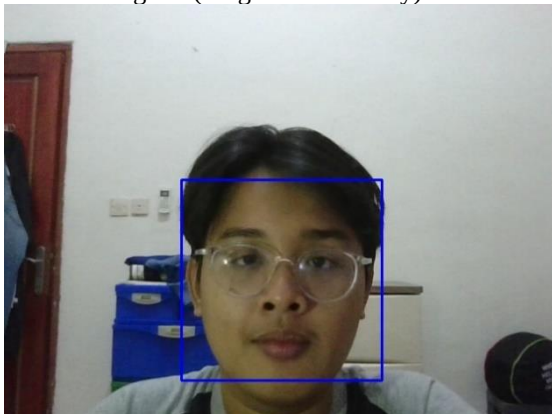
Pada penelitian ini, sistem berhasil mendeteksi dan mengenali wajah pengemudi serta plat nomor kendaraan sebagai bagian dari upaya meningkatkan keamanan di area parkir. Sistem ini dirancang untuk menyediakan metode verifikasi ganda, yang menggabungkan teknologi pengenalan wajah dan pengenalan plat nomor kendaraan. Dengan metode ini, keamanan parkir dapat ditingkatkan secara signifikan, karena setiap kendaraan yang masuk atau keluar akan diverifikasi secara otomatis oleh sistem.

Proses deteksi wajah dilakukan menggunakan algoritma Haar Cascade, yang mampu mengenali wajah manusia dengan memanfaatkan fitur-fitur tertentu dari gambar. Sementara itu, pengenalan plat nomor dilakukan menggunakan Optical Character Recognition (OCR), yang mengubah karakter pada plat nomor menjadi teks yang dapat diproses secara digital. Kombinasi kedua teknologi ini memberikan akurasi yang baik, terutama pada kondisi pencahayaan yang mendukung dan sudut kamera yang optimal.

Selain itu, sistem ini memanfaatkan metode cosine similarity untuk mencocokkan fitur wajah yang diambil saat masuk dan keluar. Dengan membandingkan vektor fitur dari gambar wajah yang telah diproses, sistem dapat menentukan tingkat kesesuaian identitas pengemudi. Proses ini dirancang agar efisien, sehingga dapat digunakan dalam skenario waktu nyata tanpa mengorbankan keakuratan.

A. Pemrosesan Wajah

1. Gambar Original (Original Face Entry):



Gambar 4.1 Foto Wajah dengan jarak $\pm 30cm$



Gambar 4.2 Foto Wajah dengan jarak $\pm 150cm$

- Pada jarak 30-150 cm dari kamera, gambar wajah menampilkan proporsi wajah yang ideal, dimana seluruh fitur wajah terlihat jelas dalam frame kamera.
- Penggunaan kacamata atau aksesoris wajah lainnya masih memungkinkan sistem untuk mendeteksi wajah selama berada pada jarak optimal.
- Akan tetapi system memerlukan cahaya eksternal karena kamera yang digunakan memiliki resolusi yang kecil

2. Hasil Preprocessing (Preprocessed Face)



Gambar 4.3 Hasil Haar Cascade dengan jarak $\pm 30cm$



Gambar 4.4 Hasil Haar Cascade dengan jarak $\pm 150cm$

- Proses preprocessing menghasilkan gambar wajah yang dinormalisasi menjadi ukuran 100x100 piksel.
- Pada jarak 30cm dari kamera, hasil preprocessing menunjukkan fitur wajah yang proporsional dan terdefinisi dengan baik, menghasilkan ekstraksi HOG yang akurat.
- Gambar dari jarak 150 cm dari kamera, ketika di-resize ke 100x100 piksel, menghasilkan fitur wajah yang kurang detail dan dapat menghasilkan nilai cosine similarity yang tidak akurat.

3. Hasil HOG

```

Press ENTER to start 10 second
timer...
Timer started! Capturing in 10
seconds...
Capturing face image now!
Saved original face as:
original_face_entry_20241219-
185023.jpg
Saved face with bounding box as:
face_with_box_entry_20241219-
185023.jpg
Saved preprocessed face as:
preprocessed_faceentry20241219-
185023.jpg
HOG Features Vector: [0.
0.          0.          ...
0.31802619 0.17191456 0.07590346]

```

Gambar 4.5 Hasil HOG dengan jarak $\pm 30cm$

```

PS C:\Users\Nopall\Documents\UNAIR
2025\VisCom\UAS Viscom> python -u
"c:\Users\Nopall\Documents\UNAIR
2025\VisCom\UAS
Viscom\face_capture.py"
Press ENTER to start 10 second
timer...
Timer started! Capturing in 10
seconds...
Capturing face image now!
Saved original face as:
original_face_entry_20241219-
185053.jpg
Saved face with bounding box as:
face_with_box_entry_20241219-
185053.jpg
Saved preprocessed face as:
preprocessed_faceentry20241219-
185053.jpg
HOG Features Vector: [0.00116659
0.          0.00058329 ...
0.24133082 0.22137944 0.03512449]
PS C:\Users\Nopall\Documents\UNAIR
2025\VisCom\UAS Viscom>

```

Gambar 4.6 Hasil HOG dengan jarak $\pm 150cm$

B. Pemrosesan License Plat:

1. Gambar Original (Original License Full):



Gambar 4.7 Foto original Plat Nomor dengan pencahayaan yang baik



Gambar 4.8 Foto original Plat Nomor dengan pencahayaan yang buruk



Gambar 4.9 Foto original Plat Nomor dengan pencahayaan yang buruk



Gambar 4.10 Foto original Plat Nomor dengan pencahayaan yang baik

- Pada kondisi pencahayaan optimal (siang hari atau pencahayaan buatan yang merata), foto original plat nomor menunjukkan detail plat nomor yang jelas dengan kontras yang baik antara karakter dan latar belakang.
- Jika pada kondisi pencahayaan kurang (cahaya redup atau bayangan), foto original plat nomor menunjukkan detail yang kurang jelas dan kontras yang rendah.

2. Hasil Cropping (Original License Cropped):



Gambar 4.11 Foto Plat Nomor yang sudah dicrop dengan pencahayaan yang buruk



Gambar 4.12 Foto Plat Nomor yang sudah dicrop dengan pencahayaan yang baik



Gambar 4.13 Foto Plat Nomor yang sudah dicrop dengan pencahayaan yang buruk



Gambar 4.14 Foto Plat Nomor yang sudah dicrop dengan pencahayaan yang baik

- Proses cropping pada pencahayaan optimal menghasilkan region of interest (ROI) yang tepat, dengan batas-batas plat nomor yang jelas teridentifikasi. Namun pada pencahayaan kurang optimal, algoritma deteksi tepi mengalami kesulitan dalam menentukan batas plat nomor, yang mengakibatkan hasil cropping yang kurang presisi namun sangat minim perbedaannya.
- Proses cropping tidak dapat maksimal karena menggunakan media smartphone untuk pengganti plat nomor sebenarnya, hal tersebut disebabkan karena bentuk dari smartphone tersebut adalah persegi panjang dan region of interest (ROI) memiliki fungsi untuk mencari bentuk persegi Panjang.

3. Hasil Preprocessing (Preprocessed License):



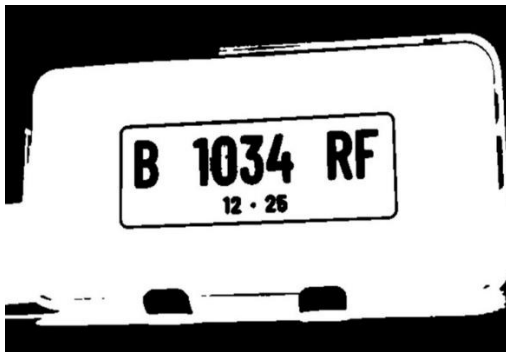
Gambar 4.13 Hasil Preprocessing Plat Nomor dengan pencahayaan yang buruk

```

Press ENTER when ready to capture
license plate image...
Capturing license plate image now!
Saved full image as:
original_license_full_20241218-
203957.jpg
Saved cropped plate as:
original_license_cropped_20241218-
203957.jpg
Saved preprocessed plate as:
preprocessed_license_20241218-
203957.jpg
No valid license plate detected
PS C:\Users\Nopall\Documents\UNAIR
2025\VisCom\UAS Viscom>

```

Gambar 4.14 Hasil program gagal membaca plat nomor



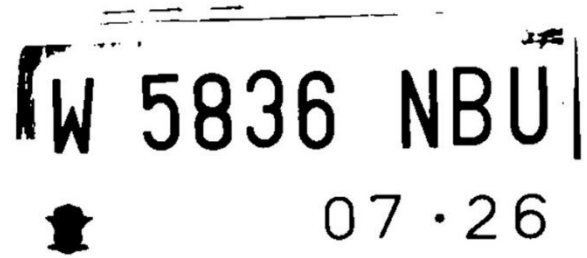
Gambar 4.15 Hasil Preprocessing Plat Nomor dengan pencahayaan yang baik

```

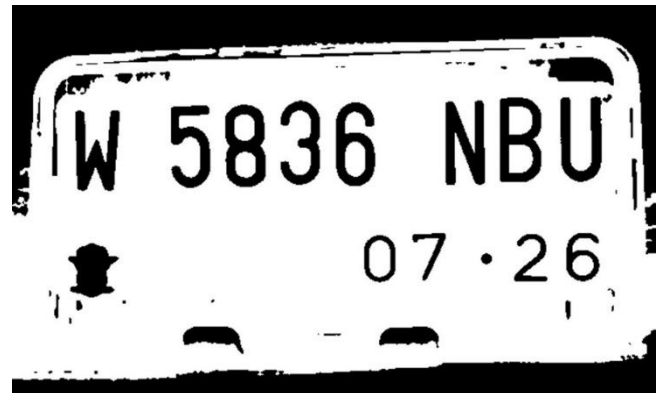
Documents\UNAIR 2025\VisCom\UAS
Viscom\plate_capture.py"
Press ENTER when ready to capture
license plate image...
Capturing license plate image now!
Saved full image as:
original_license_full_20241218-
205035.jpg
Saved cropped plate as:
original_license_cropped_20241218-
205035.jpg
Saved preprocessed plate as:
preprocessed_license_20241218-
205035.jpg
Detected license plate: B 1034 RF
Recognized license plate: B 1034
RF
PS C:\Users\Nopall\Documents\UNAIR
2025\VisCom\UAS Viscom>

```

Gambar 4.16 Hasil program berhasil membaca plat nomor



Gambar 4.17 Hasil Preprocessing Plat Nomor dengan pencahayaan yang buruk



Gambar 4.18 Hasil Preprocessing Plat Nomor dengan pencahayaan yang baik

```

Press ENTER when ready to capture
license plate image...
Capturing license plate image now!
Saved full image as:
original_license_full_20241218-
205915.jpg
Saved cropped plate as:
original_license_cropped_20241218-
205915.jpg
Saved preprocessed plate as:
preprocessed_license_20241218-
205915.jpg
Detected license plate: W 5836 NBU
Recognized license plate: W 5836
NBU
PS C:\Users\Nopall\Documents\UNAIR
2025\VisCom\UAS Viscom>

```

Gambar 4.19 Hasil program berhasil membaca plat nomor

- Tahap preprocessing pada gambar dengan pencahayaan baik menghasilkan citra biner yang

bersih dengan karakter yang terdefinisi dengan baik, memudahkan proses OCR.

- Pada gambar dengan pencahayaan buruk, hasil preprocessing menunjukkan noise yang lebih tinggi dan karakter yang kurang terdefinisi, yang dapat mengakibatkan kesalahan dalam proses OCR.
- Kesalahan dalam proses OCR salah faktornya adalah menggunakan media smartphone untuk pengganti plat nomor sebenarnya.

C. Simulasi Masuk dan Keluar Parkir

Pada Gambar 4.20, wajah pengemudi dan plat nomor kendaraan berhasil terdeteksi dan tersimpan dalam basis data saat kendaraan memasuki area parkir. Proses ini memastikan bahwa data pengemudi dan kendaraan dicatat dengan akurat.



Gambar 4.20 Wajah dan Plat yang Tersimpan Saat Masuk



Gambar 4.21 Wajah dan Plat Saat Keluar

Gambar 4.21 diatas menunjukkan data wajah dan plat nomor yang diambil ulang saat kendaraan keluar. Sistem mencocokkan data ini dengan data yang telah tersimpan untuk memverifikasi kesesuaian identitas.

```

=== Parking System Menu ===
1. Vehicle Entry
2. Vehicle Exit
3. Search Records
4. Exit Program
=====
Choose an option (1-4): 1

Initiating vehicle entry process...
Preparing to capture images in 5 seconds...
Capturing images now!
Saved face image as: preprocessed_faceentry20241211-154945.jpg
Saved license image as: preprocessed_licenseentry20241211-154946.jpg
Detected license plate: L 2094 BL
Vehicle with license plate L 2094 BL entered.

Press Enter to continue...

=== Parking System Menu ===
1. Vehicle Entry
2. Vehicle Exit
3. Search Records
4. Exit Program
=====
Choose an option (1-4): 2

Initiating vehicle exit process...
Preparing to capture images in 5 seconds...
Capturing images now!
Saved face image as: preprocessed_faceexit20241211-155008.jpg
Saved license image as: preprocessed_licenseexit20241211-155009.jpg
Detected license plate: L 2094 BL
Vehicle with license plate L 2094 BL exited. Face similarity: 0.84

```

Gambar 4.22 Hasil Program Berhasil Keluar

Gambar 4.22 menunjukkan hasil program ketika sistem berhasil memverifikasi data wajah dan plat nomor. Dengan plat nomor yang sama dan face similarity 0.84, kendaraan dapat keluar dari area parkir. Hal ini menunjukkan keandalan metode cosine similarity dalam mencocokkan fitur wajah.

Pada Gambar 4.23, sistem tetap dapat mendeteksi wajah dan plat nomor dalam kondisi pencahayaan yang berbeda, meskipun terdapat tantangan seperti sudut pengambilan gambar yang tidak optimal.



Gambar 4.23 Wajah dan Plat yang Tersimpan Saat Masuk

Gambar 4.24 menunjukkan kondisi ketika pengemudi yang berbeda mencoba mengeluarkan kendaraan. Meskipun sistem tetap mampu mendeteksi wajah dan plat nomor, wajah yang diambil saat keluar berbeda dengan data yang tersimpan saat masuk. Hal ini dirancang untuk menguji ketahanan sistem terhadap upaya pelanggaran keamanan.



Gambar 4.24 Wajah dan Plat Saat Keluar

```

=== Parking System Menu ===
1. Vehicle Entry
2. Vehicle Exit
3. Search Records
4. Exit Program
=====
Choose an option (1-4): 1

Initiating vehicle entry process...
Preparing to capture images in 5 seconds...
Capturing images now!
Saved face image as: preprocessed_faceentry20241211-155252.jpg
Saved license image as: preprocessed_licenseentry20241211-155253.jpg
Detected license plate: L 2094 BL
Vehicle with license plate L 2094 BL entered.

Press Enter to continue...

=== Parking System Menu ===
1. Vehicle Entry
2. Vehicle Exit
3. Search Records
4. Exit Program
=====
Choose an option (1-4): 2

Initiating vehicle exit process...
Preparing to capture images in 5 seconds...
Capturing images now!
Saved face image as: preprocessed_faceexit20241211-155324.jpg
Saved license image as: preprocessed_licenseexit20241211-155325.jpg
Detected license plate: L 2094 BL
Exit denied. Face does not match entry data. Similarity: 0.63

```

Gambar 4.25 Hasil Program Gagal Keluar

Gambar 4.25 menunjukkan hasil program ketika sistem gagal memverifikasi wajah pengemudi karena ketidaksesuaian data. Hal ini terjadi karena wajah yang digunakan saat keluar tidak sesuai dengan data wajah yang disimpan saat masuk. Dengan demikian, sistem menolak akses keluar untuk mencegah kemungkinan pencurian atau pelanggaran keamanan.

Secara keseluruhan, hasil penelitian ini menunjukkan bahwa sistem memiliki tingkat keberhasilan tinggi dalam kondisi ideal. Namun, terdapat kelemahan utama pada sistem, yaitu sensitivitas terhadap pencahayaan. Sistem memerlukan pencahayaan yang cukup dan konsisten agar deteksi dapat dilakukan dengan akurat. Sedikit perubahan pada kondisi pencahayaan dapat menyebabkan kegagalan deteksi wajah atau plat nomor. Untuk mengatasi hal ini, diperlukan algoritma yang lebih adaptif terhadap variasi pencahayaan, seperti penggunaan teknologi deep learning.

Selain itu, sistem ini juga dilengkapi dengan fitur pencarian data. Fitur ini memungkinkan petugas parkir untuk mencari informasi terkait kendaraan tertentu yang telah terdaftar di sistem. Misalnya, jika ada kebutuhan untuk memastikan data kendaraan atau membantu investigasi pencurian, pencarian data ini dapat dilakukan dengan mudah

dan cepat. Dengan demikian, sistem ini tidak hanya berfungsi sebagai alat pengawasan, tetapi juga menjadi solusi yang dapat diandalkan untuk mengatasi masalah keamanan, seperti angka kejadian pencurian motor yang tinggi dan keterbatasan sumber daya manusia, yang sebelumnya telah disebutkan pada bagian pendahuluan.

V. KESIMPULAN

Sistem monitoring keamanan berbasis deteksi plat nomor dan pengenalan wajah yang telah dikembangkan berhasil menyediakan metode verifikasi ganda untuk meningkatkan keamanan di area parkir. Implementasi algoritma Haar Cascade untuk deteksi wajah, OCR untuk pengenalan plat nomor, serta metode cosine similarity untuk pencocokan identitas memberikan hasil yang cukup akurat dalam kondisi ideal. Sistem ini mampu mengidentifikasi wajah pengemudi dan plat nomor kendaraan dengan baik, mencatat data dengan akurat, serta memverifikasi kendaraan yang masuk dan keluar secara efisien.

Namun, beberapa tantangan masih perlu diatasi, seperti sensitivitas terhadap pencahayaan yang tidak konsisten dan sudut kamera yang kurang optimal. Untuk meningkatkan keandalan sistem, disarankan penggunaan teknologi pembelajaran mendalam yang lebih adaptif terhadap variasi lingkungan. Selain itu, sistem ini memiliki potensi untuk dikembangkan lebih lanjut, termasuk integrasi dengan database yang lebih besar serta pengaplikasian di area publik yang lebih luas. Dengan demikian, sistem ini tidak hanya menawarkan solusi praktis untuk pengelolaan keamanan di area parkir, tetapi juga memberikan kontribusi signifikan dalam upaya mencegah kejahatan kendaraan bermotor.

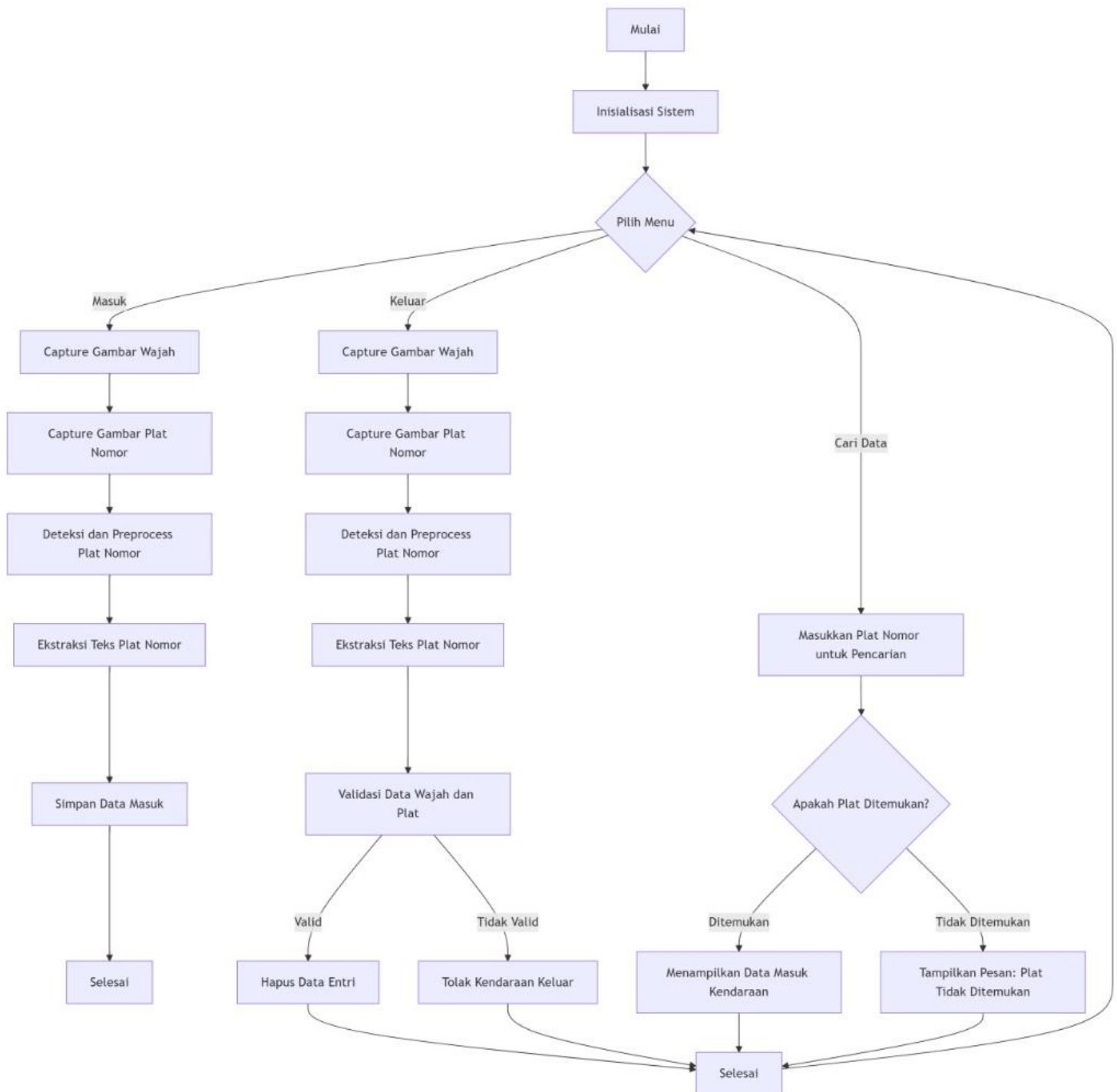
REFERENSI

- [1] Pusiknas Bareskrim Polri, "Data Kejahatan," *pusiknas.polri.go.id*. [Online]. Available: https://pusiknas.polri.go.id/data_kejahatan.
- [2] Pusiknas Bareskrim Polri, "Waspada! Pencurian Sepeda Motor Mencapai 700 Kasus dalam Dua Pekan," *pusiknas.polri.go.id*, 24 Mei 2022. [Online]. Available: <https://pusiknas.polri.go.id>.
- [3] M. A. Pradana dan I. W. Mustafa, "Design and Build a Facial Recognition-Based Motor Vehicle Security System," *International Journal of Advanced Research in Technology and Innovation*, vol. 3, no. 1, pp. 45-54, 2022.
- [4] N. Indrawan, S. A. Tjahyadi, dan R. Lestari, "Smart Security System using License Plate and Facial Recognition," *International Journal of Computer Vision and Pattern Recognition*, vol. 9, no. 3, pp. 224-232, 2024.
- [5] Shejina V, "AUTOMOTIVE THEFT DETECTION USING FACE RECOGNITION," *Int J Res Eng Technol*, vol. 05, no. 31, hlm. 47-50, Okt 2016, doi: 10.15623/ijret.2016.0531009.
- [6] Q. B. Truong, T.-L. Tran, T. T.-K. Nguyen, dan H. C. Nguyen, "Face and number plate recognition for car anti-theft," *CTU Journal of Innovation and Sustainable Development*, vol. 15, no. ISDS, hlm. 110-118, Okt 2023, doi: 10.22144/ctujoisd.2023.041.
- [7] A. D. M. . Africa, "Computer Vision on a Parking Management and Vehicle Inventory System," *International Journal of Emerging Trends in Engineering Research*, vol. 8, no. 2, hlm. 323-332, Feb 2020, doi: 10.30534/ijeter/2020/14822020.
- [8] S. Yulina, "Penerapan Haar Cascade Classifier dalam Mendeteksi Wajah dan Transformasi Citra Grayscale Menggunakan OpenCV," *Jurnal Komputer Terapan*, vol. 7, no. 1, hlm. 100-109, Jun 2021, doi: 10.35143/jkt.v7i1.3411.
- [9] V. Jabade, A. Ingale, dan R. Joshi, "Robust Face Detection and Identification using HOG-based Features and Machine Learning," dalam

- 2023 2nd International Conference on Futuristic Technologies (INCOFT), IEEE, Nov 2023, hlm. 1–6. doi: 10.1109/INCOFT60753.2023.10425077.
- [10] C. Rahmad, R. A. Asmara, D. R. H. Putra, I. Dharma, H. Darmono, dan I. Muhiqqin, “Comparison of Viola-Jones Haar Cascade Classifier and Histogram of Oriented Gradients (HOG) for face detection,” *IOP Conf Ser Mater Sci Eng*, vol. 732, no. 1, hlm. 012038, Jan 2020, doi: 10.1088/1757-899X/732/1/012038.
- [11] V. Jabade, A. Ingale, dan R. Joshi, “Robust Face Detection and Identification using HOG-based Features and Machine Learning,” dalam 2023 2nd International Conference on Futuristic Technologies (INCOFT), IEEE, Nov 2023, hlm. 1–6. doi: 10.1109/INCOFT60753.2023.10425077.
- [12] W. V. Samosir dan F. Nurzaman, “Rancang Bangun Aplikasi Circle Pertemanan Menurut Minat Dan Hoby Menggunakan Metode Cosine Similairity,” *Tekinfo Jurnal Bidang Teknik Industri dan Teknik Informatika*, vol. 24, no. 1, hlm. 52–60, Apr 2023, doi: 10.37817/tekinfo.v24i1.2795.
- [13] M. A. Irfandi, A. Romadhony, dan S. Saadah, “IMPLEMENTASI SISTEM PAKAR DIAGNOSA PENYAKIT GIGI DAN MULUT MENGGUNAKAN METODE HYBRID CASE-BASED DAN RULE-BASED REASONING,” dalam *Indonesia Symposium On Computing 2015*, SOC Press, 2015. doi: 10.21108/INDOSC.2015.19.
- [14] X. Liang dan Y. Lin, “Enhanced Parameter-Free Diversity Discriminant Preserving Projections for Face Recognition,” *Intern J Pattern Recognit Artif Intell*, vol. 32, no. 10, hlm. 1856013, Okt 2018, doi: 10.1142/S021800141856013X.
- [15] M. S. Kasem, M. Mahmoud, dan H.-S. Kang, “Advancements and Challenges in Arabic Optical Character Recognition: A Comprehensive Survey,” Des 2023.
- [16] S. Faizullah, M. S. Ayub, S. Hussain, dan M. A. Khan, “A Survey of OCR in Arabic Language: Applications, Techniques, and Challenges,” *Applied Sciences*, vol. 13, no. 7, hlm. 4584, Apr 2023, doi: 10.3390/app13074584.
- [17] R. I. Indrakusuma, A. S. Ahmadiyah, dan N. F. Ariyani, “Pengenalan dan Klasifikasi Tulisan pada Nota Pembelian Material (Studi Kasus Proyek Konstruksi),” *Jurnal Teknik ITS*, vol. 10, no. 2, Des 2021, doi: 10.12962/j23373539.v10i2.77109.
- [18] T. Octaviani, H. Setiawan, dan O. H. Kelana, “Perbandingan Pytesseract dan Template Matching Untuk Otomatisasi Input Data KTP,” *Jurnal Buana Informatika*, vol. 14, no. 02, hlm. 147–156, Okt 2023, doi: 10.24002/jbi.v14i02.7612.
- [19] Md. A. Hasnat, M. R. Chowdhury, dan M. Khan, “An Open Source Tesseract Based Optical Character Recognizer for Bangla Script,” dalam 2009 10th International Conference on Document Analysis and Recognition, IEEE, 2009, hlm. 671–675. doi: 10.1109/ICDAR.2009.62.

LAMPIRAN

- Flowchart



Gambar 6.1 Flowchart

- Kode Program

```
import cv2
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
import pytesseract
from sklearn.metrics.pairwise import cosine_similarity
import time # Add this import at the top of the file
from PIL import Image
import re
import threading
import os
import json
import pickle

# Add this line before using pytesseract
pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.exe' # Adjust the path as needed

# Replace the face model and related functions with HOG-based approach
def compute_hog(image):
    resized_image = cv2.resize(image, (128, 128))
    gray = cv2.cvtColor(resized_image, cv2.COLOR_BGR2GRAY)
    gray = cv2.equalizeHist(gray)

    cell_size = (8, 8)
    block_size = (2, 2)
    nbins = 9

    gx = cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=1)
    gy = cv2.Sobel(gray, cv2.CV_32F, 0, 1, ksize=1)
    mag, angle = cv2.cartToPolar(gx, gy, angleInDegrees=True)
    angle_bins = np.int32(nbins * angle / 360)

    h, w = gray.shape
```

```
    hist = np.zeros((h // cell_size[0], w // cell_size[1], nbins), dtype=np.float32)
    for i in range(hist.shape[0]):
        for j in range(hist.shape[1]):
            for k in range(cell_size[0]):
                for l in range(cell_size[1]):
                    y, x = i * cell_size[0] + k, j * cell_size[1] + l
                    bin_idx = angle_bins[y, x]
                    hist[i, j, bin_idx] += mag[y, x]

    hog_descriptor = []
    for i in range(hist.shape[0] - block_size[0] + 1):
        for j in range(hist.shape[1] - block_size[1] + 1):
            block_hist = hist[i:i + block_size[0], j:j + block_size[1]].ravel()
            block_hist /= np.linalg.norm(block_hist) + 1e-7
            hog_descriptor.extend(block_hist)

    return np.array(hog_descriptor)

# Replace calculate_similarity with this function
def compute_similarity_cosine(hog1, hog2):
    dot_product = np.dot(hog1, hog2)
    norm_a = np.linalg.norm(hog1)
    norm_b = np.linalg.norm(hog2)
    similarity = dot_product / (norm_a * norm_b + 1e-7)
    return similarity

# Function to preprocess image for face recognition
def preprocess_face(image):
    face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray,
```

```

scaleFactor=1.1, minNeighbors=5,
minSize=(30, 30))

    if len(faces) > 0:
        (x, y, w, h) = faces[0]
        face = image[y:y+h, x:x+w]
        face = cv2.resize(face, (100, 100))
        return face
    else:
        print("No face detected")
        return None

# Function to preprocess license plate
def preprocess_license_plate(image):
    gray = cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5,5), 0)
    thresh = cv2.threshold(blur, 0, 255,
cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)[1]

    # Find contours and select the largest
    one
    contours, _ = cv2.findContours(thresh,
cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    if contours:
        largest_contour = max(contours,
key=cv2.contourArea)
        mask = np.zeros(thresh.shape,
np.uint8)
        cv2.drawContours(mask,
[largest_contour], 0, 255, -1)
        result = cv2.bitwise_and(thresh,
thresh, mask=mask)
    else:
        result = thresh

    return result

# Add this function to detect and crop
license plate region
def detect_license_plate(image):
    # Convert to grayscale
    gray = cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY)

    # Apply Gaussian blur
    blur = cv2.GaussianBlur(gray, (5, 5),
0)

```

```

    # Apply adaptive thresholding
    thresh = cv2.adaptiveThreshold(blur,
255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH
_BINARY_INV, 19, 2)

    # Find contours
    contours, _ = cv2.findContours(thresh,
cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)

    # Initialize variables
    plate_img = None
    max_area = 0

    # Filter contours based on area and
    aspect ratio
    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)
        aspect_ratio = float(w) / h
        area = w * h

        # Filter based on aspect ratio and
        minimum area
        if 1.5 < aspect_ratio < 6 and area
        > 5000:
            if area > max_area:
                max_area = area
                # Add padding
                pad_x = int(w * 0.1)
                pad_y = int(h * 0.1)

                # Ensure coordinates are
                within image bounds
                x1 = max(0, x - pad_x)
                y1 = max(0, y - pad_y)
                x2 = min(image.shape[1], x
+ w + pad_x)
                y2 = min(image.shape[0], y
+ h + pad_y)

                plate_img = image[y1:y2,
x1:x2]

    if plate_img is not None:
        # Show the detected region (for
        debugging)

```

```

        cv2.imshow("Detected License
Plate", plate_img)
        cv2.waitKey(1000)
        return plate_img

    print("No license plate detected")
    return None

# Update the recognize_license_plate
function
def recognize_license_plate(image):
    # First detect and crop the license
    plate
    plate_img = detect_license_plate(image)
    if plate_img is None:
        return None

    # Preprocess the cropped plate
    processed =
    preprocess_license_plate(plate_img)
    processed = cv2.resize(processed, None,
fx=2, fy=2, interpolation=cv2.INTER_CUBIC)

    configs = [
        '--oem 3 --psm 6 -c
tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ
TUVWXYZ0123456789',
        '--oem 3 --psm 7 -c
tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ
TUVWXYZ0123456789',
        '--oem 3 --psm 8 -c
tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ
TUVWXYZ0123456789',
        '--oem 3 --psm 13 -c
tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ
TUVWXYZ0123456789',
    ]

    for config in configs:
        text =
        pytesseract.image_to_string(processed,
config=config)
        clean_text = re.sub(r'^A-Z0-9]',
'', text.upper())

        # Match the pattern: 1-2 letters,
1-4 numbers, 1-2 letters

```

```

        match = re.search(r'([A-
Z]{1,2})(\d{1,4})([A-Z]{1,2})', clean_text)
        if match:
            plate = '
'.join(match.groups())

            # Post-processing to ensure
correct format
            parts = plate.split()
            if len(parts) == 3 and
len(parts[0]) == 1 and len(parts[1]) <= 4
and len(parts[2]) == 2:
                final_plate = f"{parts[0]}
{parts[1]} {parts[2]}"
                print(f"Detected license
plate: {final_plate}")
                cv2.imshow("Processed
License Plate", processed)
                cv2.waitKey(1000)
                cv2.destroyAllWindows()
                return final_plate

            print("No valid license plate
detected")
            cv2.imshow("Processed License Plate",
processed)
            cv2.waitKey(1000)
            cv2.destroyAllWindows()
            return None

# Add this function to visualize face
detection
def show_detected_face(image):
    face_cascade =
cv2.CascadeClassifier(cv2.data.harcascades
+ 'haarcascade_frontalface_default.xml')
    gray = cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY)
    faces =
face_cascade.detectMultiScale(gray,
scaleFactor=1.1, minNeighbors=5,
minSize=(30, 30))

    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x+w,
y+h), (255, 0, 0), 2)

        cv2.imshow("Detected Face", image)

```

```

cv2.waitKey(0)
cv2.destroyAllWindows()

# Add this function after the
preprocess_license_plate function
def save_preprocessed_image(image,
image_type, status):
    """
    Save preprocessed image with timestamp

    Args:
        image: The preprocessed image to
save
        image_type: String indicating
'face' or 'license'
        status: String indicating 'entry'
or 'exit'
    """
    timestamp = time.strftime("%Y%m%d-
%H%M%S")
    filename =
f"preprocessed_{image_type}{status}{timesta
mp}.jpg"
    cv2.imwrite(filename, image)
    print(f"Saved {image_type} image as:
{filename}")

# Main parking system class
class ParkingSystem:
    def __init__(self):
        self.entries = {}
        self.data_dir = "parking_data"
        self.load_data() # Load saved data
when initializing

        # Create data directory if it
doesn't exist
        if not
os.path.exists(self.data_dir):
            os.makedirs(self.data_dir)
            os.makedirs(os.path.join(self.d
ata_dir, "faces"))
            os.makedirs(os.path.join(self.d
ata_dir, "plates"))

        def save_data(self):
            """Save parking entries data to
file"""

```

```

        data_file =
os.path.join(self.data_dir, "entries.pkl")
        with open(data_file, 'wb') as f:
            pickle.dump(self.entries, f)

        def load_data(self):
            """Load parking entries data from
file"""
            data_file =
os.path.join(self.data_dir, "entries.pkl")
            if os.path.exists(data_file):
                with open(data_file, 'rb') as
f:
                    self.entries =
pickle.load(f)

            def search_records(self,
license_plate=None):
                """Search for parking records"""
                if license_plate:
                    if license_plate in
self.entries:
                        return f"Found record for
license plate: {license_plate}"
                    return "No record found for
this license plate"
                else:
                    return f"Current entries:
{list(self.entries.keys())}"

            def vehicle_entry(self, face_image,
license_image):
                # Process face image
                show_detected_face(face_image.copy(
))

                face = preprocess_face(face_image)
                if face is not None:
                    save_preprocessed_image(face,
"face", "entry")
                    face_features =
compute_hog(face_image)

                # Process license plate image
separately
                plate_img =
detect_license_plate(license_image)
                if plate_img is not None:

```

```

        processed_plate =
preprocess_license_plate(plate_img)
        save_preprocessed_image(process
ed_plate, "license", "entry")
        license_plate =
recognize_license_plate(license_image)

        if face_features is not None and
license_plate:
            self.entries[license_plate] =
face_features
            return f"Vehicle with license
plate {license_plate} entered."
            elif face_features is None:
                return "Entry failed. Could not
recognize face."
            elif license_plate is None:
                return "Entry failed. Could not
recognize license plate."
            else:
                return "Entry failed. Could not
recognize face or license plate."

    def vehicle_exit(self, face_image,
license_image):
        # Process face image
        show_detected_face(face_image.copy(
))
        face = preprocess_face(face_image)
        if face is not None:
            save_preprocessed_image(face,
"face", "exit")
            face_features =
compute_hog(face_image)

            # Process license plate image
separately
            plate_img =
detect_license_plate(license_image)
            if plate_img is not None:
                processed_plate =
preprocess_license_plate(plate_img)
                save_preprocessed_image(process
ed_plate, "license", "exit")
                license_plate =
recognize_license_plate(license_image)

```

```

        if face_features is not None and
license_plate:
            if license_plate in
self.entries:
                similarity =
compute_similarity_cosine(face_features,
self.entries[license_plate])
                if similarity > 0.7:
                    del
self.entries[license_plate]
                    return f"Vehicle with
license plate {license_plate} exited. Face
similarity: {similarity:.2f}"
                else:
                    return f"Exit denied.
Face does not match entry data. Similarity:
{similarity:.2f}"
            else:
                return "Exit denied.
License plate not found in entry records."
            elif face_features is None:
                return "Exit failed. Could not
recognize face."
            elif license_plate is None:
                return "Exit failed. Could not
recognize license plate."
            else:
                return "Exit failed. Could not
recognize face or license plate."

    def capture_images():
        webcam = cv2.VideoCapture(0) # Webcam
for face
        builtin_cam = cv2.VideoCapture(1) #
Built-in camera for license plate

        if not webcam.isOpened() or not
builtin_cam.isOpened():
            print("Error: Cannot open one or
both cameras")
            return None, None

        print("Preparing to capture images in 5
seconds...")
        start_time = time.time()

        webcam_frame = None
        builtin_frame = None

```

```

def capture_webcam():
    nonlocal webcam_frame
    ret, webcam_frame = webcam.read()

def capture_builtin():
    nonlocal builtin_frame
    ret, builtin_frame =
builtin_cam.read()

while True:
    webcam_thread =
threading.Thread(target=capture_webcam)
    builtin_thread =
threading.Thread(target=capture_builtin)

    webcam_thread.start()
    builtin_thread.start()

    webcam_thread.join()
    builtin_thread.join()

    if webcam_frame is None or
builtin_frame is None:
        print("Error: Cannot read
frames from one or both cameras")
        webcam.release()
        builtin_cam.release()
        return None, None

    elapsed_time = int(time.time() -
start_time)
    remaining_time = 5 - elapsed_time

    if remaining_time <= 0:
        break

    # Add countdown text to the frames
    cv2.putText(webcam_frame,
f"Capturing in: {remaining_time}s", (10,
30),
cv2.FONT_HERSHEY_SIMPLE
X, 1, (0, 255, 0), 2)
    cv2.putText(builtin_frame,
f"Capturing in: {remaining_time}s", (10,
30),
cv2.FONT_HERSHEY_SIMPLE
X, 1, (0, 255, 0), 2)

```

```

    cv2.imshow("Webcam (Face)",
webcam_frame)
    cv2.imshow("Built-in Camera
(License Plate)", builtin_frame)

    if cv2.waitKey(1) & 0xFF ==
ord('q'):
        webcam.release()
        builtin_cam.release()
        cv2.destroyAllWindows()
        return None, None

    print("Capturing images now!")
    ret_webcam, final_webcam_frame =
webcam.read()
    ret_builtin, final_builtin_frame =
builtin_cam.read()

    webcam.release()
    builtin_cam.release()
    cv2.destroyAllWindows()

    if not ret_webcam or not ret_builtin:
        print("Error: Cannot capture images
from one or both cameras")
        return None, None

    return final_webcam_frame,
final_builtin_frame # Return face image
first, then license plate image

# Add this function for the menu system
def display_menu():
    print("\n=== Parking System Menu ===")
    print("1. Vehicle Entry")
    print("2. Vehicle Exit")
    print("3. Search Records")
    print("4. Exit Program")
    print("=====")
    return input("Choose an option (1-4):
")

# Modify the main execution part
def main():
    parking_system = ParkingSystem()

    while True:

```



```

        choice = display_menu()

        if choice == '1':
            print("\nInitiating vehicle
entry process...")
            entry_face_image,
entry_license_image = capture_images()

            if entry_face_image is not None
and entry_license_image is not None:
                cv2.imshow("Entry Face
Image", entry_face_image)
                cv2.imshow("Entry License
Plate Image", entry_license_image)
                cv2.waitKey(0)
                cv2.destroyAllWindows()

                print(parking_system.vehicl
e_entry(entry_face_image,
entry_license_image))
            else:
                print("Failed to capture
entry images")

        elif choice == '2':
            print("\nInitiating vehicle
exit process...")
            exit_face_image,
exit_license_image = capture_images()

            if exit_face_image is not None
and exit_license_image is not None:
                cv2.imshow("Exit Face
Image", exit_face_image)
                cv2.imshow("Exit License
Plate Image", exit_license_image)
                cv2.waitKey(0)
                cv2.destroyAllWindows()

                print(parking_system.vehicl
e_exit(exit_face_image,
exit_license_image))
            else:
                print("Failed to capture
exit images")

        elif choice == '3':

```

```

        search_option = input("\nEnter
license plate to search (or press Enter to
list all): ")
        print(parking_system.search_rec
ords(search_option))

        elif choice == '4':
            print("\nThank you for using
the Parking System!")
            break

        else:
            print("\nInvalid option! Please
choose 1-4.")

            input("\nPress Enter to
continue...")

if __name__ == "__main__":
    main()

```

- Dokumentasi

